ChargeSight — Validation & Scoping Research

Oct 23, 2026 · @Pratham

1. Executive summary and recommendation

Recommendation: proceed, but revise the scope. A charger-monitoring dashboard with alerts and tickets is already sold by many vendors, so it cannot be our contribution. What we can defend is narrower: an open, explainable fault-prioritisation method for small operators, tested against first-come-first-served in a reproducible simulation.

- **The problem is real and documented in India.** Only about 71% of FAME-II-approved public stations were operational on 1 March 2026 (6,645 of 9,332). IEEFA's September 2026 report cites a 2024 Delhi sample in which nearly 84% of chargers were not functional, and recommends uptime mandates and maximum fault-resolution times.
- **Monitoring, alerts, remote recovery and ticketing are commonly available.** Driivz, Monta, AMPECO, Ampcontrol, ChargerHelp and Indian platforms (Statiq, Numocity, Bolt.Earth) all document some of these.
- **Prioritisation exists but is opaque in public material.** Driivz claims "intelligent priority routing" and Ampcontrol sorts tickets by priority, but the public pages we read do not say how priority is computed or explain individual decisions.
- **Our contribution:** a transparent priority score that shows why each charger is ranked where it is, plus a policy simulator that measures what that ranking does to availability and downtime cost under stated assumptions.
- **What we must not claim:** that we monitor real chargers, that competitors lack features we did not find, or any improvement before the experiment in Section 9 is actually run.

2. Current industry practice and competitor research

Commercial charge-point management systems (CPMS) already cover monitoring, alerting and most of the maintenance loop. Everything below is what each vendor documents publicly; vendor numbers are marketing claims unless marked otherwise.

Product	What its public pages document	Quantified claims (vendor unless noted)
Driivz (Vontier)	Real-time automated issue detection, Alert Management System, self-healing remote resolution, proactive ticket generation, "intelligent priority routing", technician mobile interface, predictive models on power, temperature and connector wear	"Up to 80%" of issues resolved remotely (EV Report); 150,000+ chargers managed; "guaranteed 99.9% availability" (Driivz)
Monta	17 pre-built alert types (disconnect > 15 min, error states, OCPP security events, failed sessions); driver- and support-raised alerts; per-site SLAs; Self-heal per charger model and firmware; Workflows that reboot, escalate or send a work order to the operator's field-service system; filter by priority	Survey: only 12% of CPOs find failed-session causes "extremely easy" to identify; almost a third unsure (Fleet World, sample size not stated)
AMPECO	Real-time monitoring and diagnostics, failure alerts, remote start/stop/unlock/availability, auto-fault recovery, view when and why a failure occurred; Jira ticketing via marketplace app	None found
Ampcontrol	Automatic ticket creation, sort by category or priority, SLA tracking against uptime goals, assignment by technician availability and expertise; Alert Center with urgency, suggested actions, per-charger failure statistics	None found
ChargerHelp	EMPWR software: real-time diagnostics from field data plus OCPP error codes, predictive analytics, uptime reports; Reliability-as-a-Service field technicians	Up to 16% of chargers report a false status (its own 2024 report, method not shown)
ChargePoint	Sells a service plan (Assure) for maintenance; plan terms could not be retrieved (documentation behind login)	Not verified

Product	What its public pages document	Quantified claims (vendor unless noted)
Statiq CSMS (India)	Real-time charger monitoring, OCPP-compliant (version not named), alerts, "Error & Complaint Management", dashboards	None found
Numocity (India)	Real-time monitoring, fault management, remote diagnostics, analytics; holds an OCA CSMS certificate	None found
Bolt.Earth CMS (India)	Real-time health monitoring, third-party OCPP chargers, remote session control, warranty and service-ticket tracking, firmware updates, ESG reports	None found

Tata Power EZ Charge and ChargeZone publish driver-facing apps; we found no public documentation of their internal maintenance tooling, so they are listed as potential customers, not as competitors.

Independent evidence on reliability (not vendor claims):

- A 2022 UC Berkeley-affiliated field study of all 657 CCS connectors at 181 Bay Area fast-charging stations found 72.5% functional and 22.7% non-functional, against 95–98% uptime reported by providers. The authors blame inconsistent uptime definitions (Rempel et al., arXiv).
- J.D. Power's 2025 study, as cited by Driivz, puts failed charging attempts at 14%, down from 19% in 2024.
- The US NEVI rule requires 97% average annual uptime per port, excluding outages from maintenance, vandalism and natural disasters (summary). We did not find an equivalent Indian mandate; IEEFA recommends one.

3. Feature comparison matrix

Ten of the thirteen features below already exist in at least two products; three are not confirmed in any public documentation we read. "Documented" means the vendor's own page says so; a blank means we did not find it, not that the product lacks it.

Feature	Products that document it	Classification
Real-time charger status	Driivz, Monta, AMPECO, Ampcontrol, ChargerHelp, Statiq, Numocity, Bolt.Earth, SteVe, CitrineOS	Commonly available
OCPP connectivity and event logging	Statiq, Numocity (OCA-certified), Bolt.Earth, ChargerHelp, Monta, SteVe, CitrineOS	Commonly available
Automatic fault and offline alerts	Driivz, Monta (disconnect > 15 min), AMPECO, Ampcontrol, Statiq	Commonly available
Remote recovery or self-healing	Driivz, Monta, AMPECO	Available in some
Maintenance tickets or work orders	Ampcontrol, Driivz, Bolt.Earth, Monta (to external system), AMPECO (Jira)	Available in some
Technician assignment and dispatch	Ampcontrol, Driivz, ChargerHelp (own field team)	Available in some
SLA tracking	Ampcontrol, Monta	Available in some
Uptime / availability reporting	ChargerHelp, Ampcontrol (against uptime goals)	Available in some
Fault priority ranking	Driivz, Ampcontrol, Monta (filter)	Available in some; method not public
Failure prediction	Driivz, ChargerHelp	Available in some; vendor claims only
Explanation of why a fault was ranked where it is	—	Not confirmed by public documentation
Side-by-side comparison of maintenance policies on repeatable simulated events	—	Potential opportunity
Downtime cost estimate with explicit, editable assumptions	—	Potential opportunity

Sources are linked per product in Section 2. Large vendors may offer the last three privately or to enterprise customers; the gap is in public, inspectable methods, not proven absence.

4. Evidence-based gap analysis

The gap is not missing software; it is missing transparency and evidence for small operators. Three findings support a narrow scope.

1. **Operators struggle to diagnose, even with tools.** In Monta's survey, almost a third of CPOs were unsure how to find root causes of failed sessions. Reported uptime and field-measured functionality also diverge (95–98% claimed vs 72.5% measured in the Bay Area study), partly because uptime is defined inconsistently.
2. **Prioritisation is claimed but not explained publicly.** Driivz and Ampcontrol describe priority routing and sorting, but not the factors or weights. A small operator cannot audit why charger A was fixed before charger B.
3. **Indian policy is moving toward measured uptime.** IEEFA recommends linking public support to verified uptime and maximum fault-resolution times. If that happens, operators will need a defensible, documented way to decide repair order and report downtime.

Candidate gaps tested against the evidence:

Candidate	Already offered?	Feasible for students?	Verdict
Transparent fault-priority ranking	Ranking yes; explanation not found	Yes, rule-based	Keep as core
Compare maintenance policies on repeatable events	Not found	Yes, discrete-event simulation	Keep as core (the evaluation)
Explain each ranking decision	Not found	Yes, show factor contributions	Keep, part of the score
Effect of response delay on availability	Not found as a public tool	Yes, sweep technician count and travel time	Keep as sensitivity analysis
Downtime cost with explicit assumptions	Not found	Yes, but numbers are assumptions	Keep, clearly labelled
Failure prediction with ML	Claimed by Driivz, ChargerHelp	No real failure data to train on	Drop; mention as future work

Candidate	Already offered?	Feasible for students?	Verdict
Full CPMS replacement	Many vendors	No	Drop

A useful honesty check: with identical repair times, a work-conserving queue gives the same **total** waiting time under any service order (Kleinrock's conservation law). Priority can only help on **weighted** outcomes, such as downtime at busy sites or high-severity waits, and usually at the cost of longer waits for low-priority faults. The experiment must measure both sides.

5. Recommended title and problem statement

Title: ChargeSight: Explainable Fault Prioritisation and Downtime-Impact Simulation for Small EV Charge-Point Operators

Problem statement: Indian public EV chargers are frequently non-functional; only about 71% of FAME-II-approved stations were operational in March 2026. Small and mid-sized charge-point operators with few technicians must decide which faulted or offline charger to repair first, usually in order of arrival or by judgement. Commercial platforms advertise priority routing, but their ranking methods are not publicly documented, so an operator cannot inspect or justify the order. ChargeSight will (a) rank open faults with a transparent, rule-based score whose every factor is shown to the operator, and (b) measure, in a reproducible simulated fleet, how that ranking changes availability, weighted downtime and waiting time compared with first-come-first-served handling.

One-line pitch: Know which charger to fix first, and why.

6. Scope

The build has one core (the priority engine and its evaluation) and a thin demo layer around it.

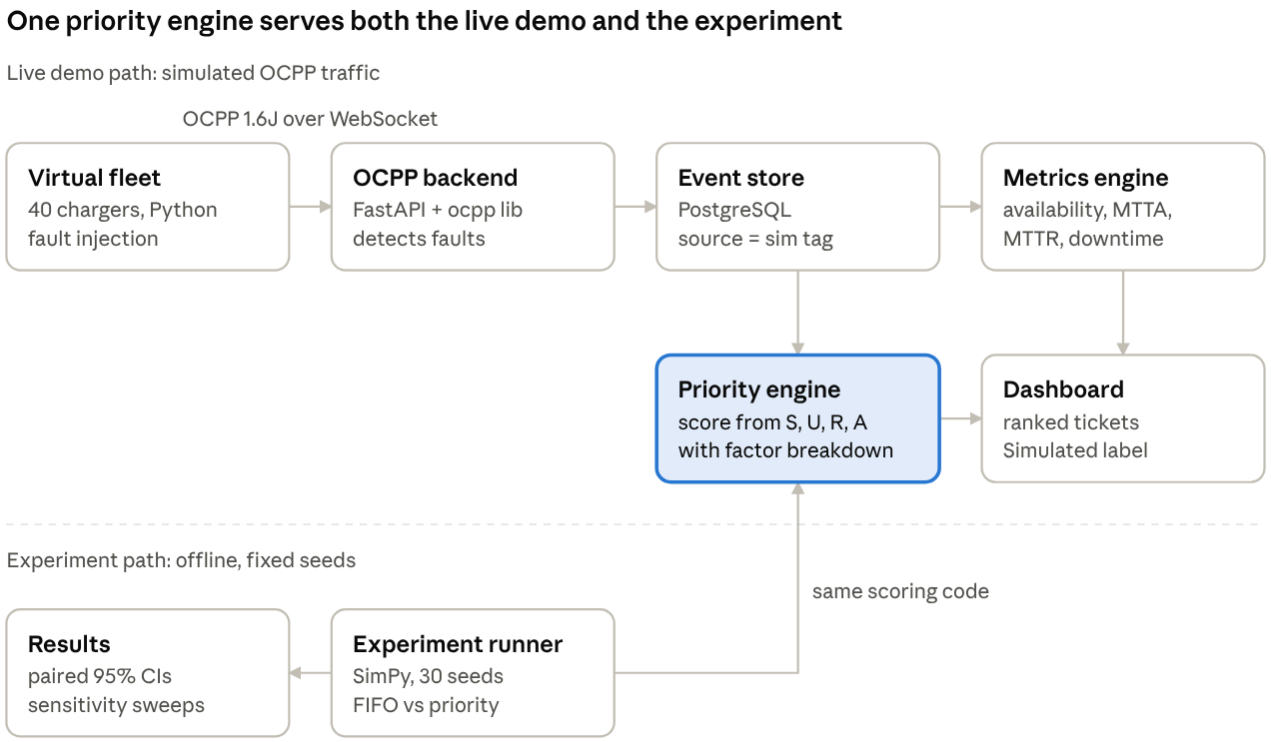
Included	Purpose
Virtual fleet of 40 chargers across 6 Bengaluru sample sites	Realistic context for the demo and the experiment
OCPP 1.6J subset: BootNotification, Heartbeat, StatusNotification, MeterValues, Start/StopTransaction, Reset	Shows protocol realism in the live demo

Included	Purpose
Fault injection: HighTemperature, OverCurrentFailure, GroundFailure, missed heartbeats, payment/reader faults	Exercises every priority factor
Fault and offline detection from the event stream	Opens tickets automatically
Explainable priority score with per-factor breakdown	Core contribution
Ticket workflow: open, assigned, en route, on site, resolved	Measures acknowledge and repair times
Metrics: availability, downtime hours, MTTA, MTTR, weighted downtime, energy delivered	Fleet view and experiment outputs
Policy simulator: FIFO vs priority, fixed seeds, parameter sweeps	Core evaluation
Downtime cost estimate from editable assumptions	Business case, clearly labelled
Dashboard with a permanent "Simulated data" label	Demo and pitch

Excluded: real charger or operator integration; payments and roaming (OCPI); ML failure prediction (no real failure data); smart charging and energy management; mobile app; OCPP 2.0.1 and security profiles; production hardening (authentication, multi-tenancy, scaling).

7. Technical architecture and simulation plan

A full OCPP implementation is not needed for the evaluation; a small OCPP 1.6J subset is enough for the demo, and the experiment runs as a separate discrete-event simulation that reuses the same priority code.



ChargeSight architecture · demo path and experiment path

Simulated chargers talk real OCPP to the backend for the live demo; the experiment runner bypasses the network and replays fault traces directly into the same priority engine, which keeps runs fast and repeatable.

OCPP 1.6J messages used

Message	Direction	Used for
BootNotification	Charger → backend	Register charger; detect reboot after a remote reset
Heartbeat	Charger → backend	Offline detection (no heartbeat for 3 intervals)
StatusNotification	Charger → backend	Available, Charging, Faulted, Unavailable + `errorCode`
MeterValues	Charger → backend	Power, energy, connector temperature
StartTransaction / StopTransaction	Charger → backend	Sessions, energy delivered, site utilisation (U)
Reset	Backend → charger	Remote recovery attempt before dispatch

Generating behaviour. Each virtual charger is an asyncio task with a state machine: sessions start and stop with site-specific arrival rates; faults arrive as a Poisson process with the mix in Section 9; comms loss means the task stops sending heartbeats for a random period; recovery happens after a successful remote reset or a completed ticket. Every random draw comes from a seeded generator, so a seed reproduces the run exactly.

Storing events and computing metrics. Store every message once in an append-only `events` table (timestamp, charger, type, payload, `source = 'sim'`, seed). Derive `status_intervals` from consecutive status changes. Then:

- Availability = time in Available or Charging ÷ observed time, per port.
- Downtime = time in Faulted, Unavailable or Offline; Offline starts at the last heartbeat, not at detection.
- MTTA = assigned − detected; MTTR = resolved − detected, per ticket.
- Energy delivered = sum of (meterStop − meterStart) per transaction.

State the uptime definition and exclusions in the report; the Bay Area study shows mismatched definitions are a main source of inflated uptime claims.

Keeping simulated data obvious. A permanent "Simulated data" banner, the seed and generator version in the dashboard footer, `source = 'sim'` on every stored event, and charger IDs from a reserved prefix (for example `SIM-KRM-01`).

8. GitHub references and setup feasibility

Use the MIT-licensed `ocpp` Python library for both the virtual chargers and our backend; keep the GPL and heavyweight systems as references only. Licences and last-commit dates below were checked by cloning each repository on 23 Oct 2026; we did not run full builds.

Repository	Licence	What it is	Setup per its README	Fit for us
mobilityhouse/ocpp	MIT	Python library for OCPP-J messages; 1.6 and 2.0.1, 2.1 added in v2.1.0 (14 Jul 2025); last commit 23 Oct 2026	`pip install ocpp`; you write the WebSocket server and handlers	**Use.** Library, not a full CSMS; our backend and our simulated chargers both build on it

Repository	Licence	What it is	Setup per its README	Fit for us
solidstudiosh/ocpp-virtual-charge-point	Apache-2.0	Node.js terminal charge-point simulator, 1.6 / 2.0.1 / 2.1; admin WebSocket to send arbitrary messages; last commit 28 Aug 2026	`npm install`, set `WS_URL`, `CP_ID`, run `npm start index_16.ts`	**Optional.** One process per charger; good for a cross-check that our backend accepts a third-party client
steve-community/steve	GPL-3.0	Java OCPP central system, 1.6J and 1.6S; web UI; last commit 11 Oct 2026	MySQL/MariaDB (UTC time zone), Maven build, or `docker compose up -d` → port 8180	**Reference.** Full CSMS we would have to modify in Java; GPL obligations if we distribute changes
citrineos/citrineos-core	Apache-2.0	TypeScript CSMS, OCPP 1.6 and 2.0.1; RabbitMQ, PostgreSQL, Hasura, Next.js operator UI; last commit 21 Oct 2026	Docker stack launcher	**Reference.** Production-grade but heavy for a 9-week student build
matth-x/MicroOcppSimulator	GPL-3.0	GUI simulator for the MicroOcpp client; last commit 3 Nov 2024	`docker build`, run on port 8000	**Reference.** One charger with a GUI; less active

Known limitations: none of these generate realistic fault patterns on their own; fault timing and type come from our own simulator. A single-process Python simulator running 40 chargers as asyncio tasks is lighter than 40 separate Node.js processes.

9. Evaluation methodology

Run both policies on the identical fault trace, 30 seeds each, and judge the priority policy on weighted downtime and high-severity waiting time, not on total downtime. All parameter values below are assumptions chosen to span light to heavy load; we found no published Indian charger failure rates.

Priority score (proposed policy). Each open fault gets a score from four visible factors, each scaled 0–1:

```
P = 0.4\,S + 0.3\,U + 0.2\,R + 0.1\,A
```

S is severity (cannot charge = 1, degraded = 0.6, cosmetic = 0.2). U is the site's normalised daily sessions. R is capacity lost at the site (1 if it was the last working fast charger there). A is ageing, wait time divided by a 24-hour cap, so low-priority faults cannot starve. The ticket shows each term's contribution, for example "S 0.40 + U 0.21 + R 0.20 + A 0.03 = 0.84". Weights are tested in the sensitivity runs.

Baseline policy: first-come-first-served by fault detection time.

Parameter	Base value	Sensitivity range
Fleet	40 chargers, 6 sites, mixed DC/AC	—
Period	30 simulated days per run	—
Replications	30 seeds, same seeds for both policies	—
Fault arrivals	Poisson, 0.05 per charger-day	0.02, 0.10
Fault mix	Ground 15%, over-current 15%, high temp 20%, comms loss 30%, reader/payment 20%	± 10 points per class
Remote reset success	Comms loss 60%, reader 40%, others 0%	± 20 points
Technicians	2, one job at a time	1, 3
Travel time	20–60 min by zone	× 0.5, × 2
Repair time	Lognormal, mean 45–180 min by fault class	× 0.5, × 2
Priority weights	0.4 / 0.3 / 0.2 / 0.1	Equal weights; severity-only

Metrics:

Metric	Definition
Fleet availability	Port-time in Available or Charging ÷ total port-time
Total downtime	Sum of charger-hours not available
Weighted downtime	Downtime × site utilisation weight (proxy for lost sessions)
MTTA	Mean time from detection to technician assigned
MTTR	Mean time from detection to resolved
High-severity wait	Mean and 95th-percentile wait for faults with S = 1
Worst-case low-priority wait	Maximum wait for faults with S ⩽ 0.6 (starvation check)

Fairness and reproducibility: generate each seed's fault trace once and replay it under both policies (common random numbers); publish seeds, parameters and code; report the paired difference per metric with a 95% confidence interval over 30 seeds; fix the policy and weights before looking at results.

Decision rules:

- **Supports the proposal:** the 95% CI of the paired difference in weighted downtime and high-severity wait lies below zero in the base case and in most sensitivity cases, and worst-case low-priority wait stays under the 24-hour ageing cap.
- **Rejects it:** no significant difference, gains that disappear under most sensitivity settings, or starvation beyond the cap.
- **Expected and acceptable:** total downtime roughly unchanged (conservation law, Section 4).

Claims allowed now: the design, the score's transparency, and that it is testable. **Claims that need the experiment:** any percentage improvement. **Claims never allowed:** that results transfer to real fleets without real operator data.

10. Market, customer and business model

The best first customer is a small or mid-sized Indian operator or maintenance contractor that already has a CPMS but no transparent way to order repairs; ChargeSight should plug into that CPMS, not replace it.

Verified facts

- India had 29,151 public charging stations by December 2025; about 71% of FAME-II-approved stations were operational on 1 March 2026 (Bolt.Earth, citing MHI data).
- IEEFA (Sept 2026) recommends uptime mandates, maximum fault-resolution times and public availability data (report).
- Indian CPMS vendors (Statiq, Numocity, Bolt.Earth) publish monitoring and alerts; we found no public description of explainable repair prioritisation from them.

Reasonable assumptions (to validate with interviews)

- Small operators run a few technicians across many sites, so repair order matters.
- They will pay for a decision tool only if it plugs into their existing CPMS through an API or CSV export.
- Uptime reporting requirements, if introduced, would raise demand for auditable downtime records.

Segment	Their problem	Why ChargeSight
Small and mid-sized CPOs (tens to a few hundred chargers)	Few technicians, many sites, ad-hoc repair order	Ranked, explained queue; downtime report
EVSE maintenance contractors	Must justify response order to several clients	Auditable priority log per client
Fleet depots (e-bus, e-3W, delivery fleets)	A dead charger strands vehicles	Weight U and R by fleet schedule
Large CPOs	Already have Driivz-class tools	Not a target

Alternatives customers use today: their CPMS alert list, WhatsApp groups with technicians, spreadsheets, or an enterprise platform with built-in routing.

Value proposition: fix the charger that costs the most first, show why, and prove it with a downtime report.

Revenue options (no prices invented): per-charger monthly subscription; per-site licence for maintenance contractors; one-time policy audit using the simulator on an operator's own historical logs. Pricing must come from customer interviews.

Business Model Canvas (draft)

Block	Content
Customer segments	Small/mid CPOs, maintenance contractors, fleet depots
Value propositions	Explainable repair order, policy what-if simulation, downtime cost report
Channels	Direct outreach, CPMS integration partners, EV industry events
Customer relationships	Pilot on historical logs, then subscription with onboarding
Revenue streams	Per-charger subscription, contractor licence, audit fee
Key resources	Priority engine, simulator, OCPP know-how
Key activities	Integration adapters, model tuning, reporting
Key partners	CPMS vendors with open APIs, maintenance contractors
Cost structure	Cloud hosting, development, sales, support

11. Budget, timeline, risks and PME deliverables

The project needs about 44 working days and roughly ₹7,150 in cash; everything else is team time. Team size (4) and all rupee figures are planning assumptions to replace with your own.

Schedule and critical path (working days, computed with CPM)

ID	Activity	Days	Depends on	ES	EF	Float
A	Problem and competitor research	5	—	0	5	0
B	Requirements and scope freeze	3	A	5	8	0
C	Architecture and data model	4	B	8	12	0
D	Fleet and fault simulator	8	C	12	20	3
E	Backend ingestion and event store	6	C	12	18	0
F	Uptime and MTTR metric engine	4	E	18	22	4
G	Priority score and explanation engine	5	E	18	23	0
H	Policy simulator (FIFO vs priority)	6	D, G	23	29	0

ID	Activity	Days	Depends on	ES	EF	Float
I	Dashboard UI	7	F	22	29	4
J	Experiment runs and analysis	4	H	29	33	0
K	Market, cost, BMC write-up	6	B	8	14	23
L	Integration and testing	4	I, J	33	37	0
M	Report and pitch deck	5	K, L	37	42	0
N	Rehearsal and submission	2	M	42	44	0

Critical path: **A → B → C → E → G → H → J → L → M → N = 44 days.** Any slip in the priority engine (G) or the policy simulator (H) delays submission; the dashboard (I) has 4 days of slack.

Cash budget (assumptions)

Item	Cost (₹)
Cloud hosting beyond free tiers	2,000
Domain name, 1 year	1,000
Customer interviews (travel)	2,000
Poster and printing	1,500
Contingency (10%)	650
Total cash	**7,150**

In-kind labour for the PME cost sheet: 4 people × 9 weeks × 12 h = 432 person-hours; at a notional ₹300/h that is ₹1,29,600. No hardware is purchased.

Risk register

Risk	Likelihood	Impact	Mitigation
No real operator data	High	Medium	Present results as simulation-only; offer the audit to one operator as future work
Experiment shows no gain	Medium	Medium	Pre-registered decision rules; a negative result is still a valid finding

Risk	Likelihood	Impact	Mitigation
Simulation assumptions challenged in viva	High	Medium	Publish parameters, run sensitivity sweeps, cite where values are assumed
OCPP integration takes longer	Medium	Medium	Demo layer is off the critical path; fall back to an internal event bus
Scope creep into a full CPMS	Medium	High	Section 6 exclusions frozen at activity B
Team time lost to placements	High	High	Float on D, F, I; weekly check-ins; owners per activity
Simulated data mistaken for real	Low	High	Permanent label, `source = sim` on every event, seed shown in UI
Security if ever deployed	Low	High	Out of scope; production needs OCPP security profiles and TLS

Where each PME deliverable comes from

PME item	Source in this doc
1–2 Problem, objectives, scope	Sections 4–6
3 Methodology	Sections 7 and 9
4 WBS and activities	Activity table above (group A–C planning, D–J build, K–N close)
5 Network diagram and critical path	Draw from the Depends-on column; path above
6 Gantt chart	Bars from ES/EF above
7 Resources	4 students, laptops, free-tier cloud, open-source libraries
8 Cost and budget	Cash budget and in-kind labour
9 Risk plan	Risk register
10 Business Model Canvas	Section 10
11 Prototype / simulation	Dashboard + policy simulator
12 Market analysis	Section 10

PME item	Source in this doc
13 Expected outcomes	Section 9 decision rules (no results claimed in advance)
14 Pitch	"Know which charger to fix first, and why" + experiment results

12. Final decision

Proceed with the revised scope. Keep the topic; change the contribution from "a monitoring dashboard" to "an explainable repair-priority method, evaluated against first-come-first-served in a reproducible simulation".

- **Why not drop it:** the problem is current and documented in India, the build needs no hardware, and the experiment gives the pitch real numbers of our own instead of borrowed claims.
- **Why not keep the original scope:** monitoring, alerts, remote recovery and ticketing are commonly available; presenting them as new would not survive a viva.
- **Before building:** freeze scope (activity B), agree the priority factors and weights, and try to speak to one operator or maintenance contractor to test the assumptions in Section 10.

Sources

- Bolt.Earth: India's EV charger gap, 71% operational
- IEEFA: EV charging experience in India (Sept 2026)
- Rempel et al.: Reliability of open public DC fast chargers (arXiv 2203.16372)
- NEVI final rule summary (Zeta)
- Driivz: proactive maintenance blog · Driivz operational excellence · EV Report on Driivz
- Monta: alerting · Fleet World on Monta survey
- AMPECO: remote management and maintenance · AMPECO Jira app
- Ampcontrol: ticket management · Ampcontrol Alert Center
- ChargerHelp: reliability post
- Statiq CSMS · Numocity CPMS (Siemens) · Numocity OCA certificate · Bolt.Earth CMS
- mobilityhouse/ocpp · ocpp-virtual-charge-point · SteVe · CitrineOS · MicroOcppSimulator